

MARS ROVER MANIPAL

RESEARCH AI TASKPHASE

Report 9

Adarsh Inaganti

240905294

adarshinaganti006@gmail.com

[GitHub repo](#)

1. Review questions

1.1 What is a scree plot?

A scree plot is a simple graph used in PCA (Principal Component Analysis) and factor analysis to decide how many components to keep.

What it represents

- **X-axis** represents the component number (PC1, PC2,...)
- **Y-axis** represents the eigenvalue or explained variance ratio of each component.
- The plot usually drops sharply at first, then flattens.

How to read the graph

- We need to identify the point where the curve stops dropping steeply (elbow).
- Components before the elbow are meaningful.
- Components after the elbow contribute very little and are usually discarded.

Why it's used

- Helps avoid keeping too many components.
- Visual way to balance dimension reduction vs information retained.

1.2 What is the loss function of SVM?

The loss function of SVM is the **hinge loss** function.

$$L_i = \max(0, 1 - y_i(w^T x_i + b))$$

However, the SVM minimizes hinge loss and regularization.

$$\min_{w,b} \frac{1}{2} ||w||^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b))$$

1.3 What is precision and recall?

Precision

It shows how many of the predicted positives were actually positive.

$$Precision = \frac{TP}{TP+FP}$$

Recall

It shows how many of the actual positives the model correctly found.

$$Recall = \frac{TP}{TP+FN}$$

1.4 Why do we also need F1 score or precision or recall? Why is accuracy alone not sufficient as a metric?

Accuracy can be high even if the model is unoptimized.

Example

- Suppose 95% of samples are negative.
- A model which predicts always negative gets:
 - Accuracy = 95%
 - Precision = 0, recall = 0
- Accuracy hides the failure.

Accuracy treats all mistakes equally.

Example

- Missing a cancer patient (FN) is worse than a false alarm (FP).
- Spam filters: FP (important mail goes to spam) is worse than FN.
- Precision and recall fix these issues by measuring which type of error matters.

Precision and recall both focus on positive class performances.

Example

- Accuracy averages performance across both classes.
- Precision/recall zoom in on the positive class.
- This is critical in fraud detection, medicine, intrusion detection, etc.

F1 score balances precision and recall

- This is because F1 is the harmonic mean of precision and recall.
- Useful when:
 - Classes are imbalanced
 - Both FP and FN matter

2. K-nearest neighbour (KNN)

2.1 Train test split

```
1 # train test split (80/20)
2
3 def train_test_split(X, y, test_size=0.2, random_state=1):
4     rng = np.random.RandomState(random_state)
5     idx = np.arange(X.shape[0])
6     rng.shuffle(idx)
7     split = int(X.shape[0] * (1 - test_size))
8     train_idx, test_idx = idx[:split], idx[split:]
9
10    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
```

- The function takes the following parameters:
 - X: feature array
 - y: label/target array
 - test_size=0.2: 20% split of train/test
 - random_state=1: seed for random number generator
- The rng object is a pseudorandom number generator seeded with random_state.
- The index array idx has n rows/samples in X.
- The array is shuffled in-place using the seeded rng. idx is now a permutation of random indices.
- The split variable stores the result of calculation of the split point. This is typecasted to an integer.
- The train_idx and test_idx variables are the shuffled idx arrays which are sliced into the first split indices and the remaining indices respectively.
- The function returns four arrays:
 - X[train_idx]: training features
 - X[test_idx]: test features
 - y[train_idx]: training targets
 - y[test_idx]: test targets

2.2 Defining the scaler

```
1 # scaler
2
3 class StandardScaler:
4     def fit(self, X):
5         self.mean_ = X.mean(axis=0)
6         self.std_ = X.std(axis=0, ddof=0)
7         self.std_[self.std_ == 0] = 1
8
9     def transform(self, X):
10        return (X - self.mean_) / self.std_
11
12    def fit_transform(self, X):
13        self.fit(X)
14        return self.transform(X)
```

The StandardScaler class has three functions:

- **fit()**
 - Takes a parameter X (feature array).
 - Computes mean and standard deviation.
 - Replaces any 0 deviation with 1 to avoid division by zero error.
- **transform()**
 - Takes a parameter X (feature array).
 - Applies feature-wise standardization using z-score.
- **fit_transform()**
 - Takes a parameter X (feature array).
 - Used to apply fit and transform in one step.

2.3 Calculating distances

```
1 # distance
2
3 def euclidean_distances(X_train, x_query):
4     diff = X_train - x_query
5     return np.sqrt(np.sum(diff**2, axis=1))
```

- The function `euclidean_distances()` takes two parameters:
 - `X_train`: a matrix of shape `(n_samples, n_features)`
 - `x_query`: a single row vector of shape `(n_features,)`
- It returns a 1D array of length `n_samples`, where each element is the distance from `x_query` to a point in `X_train`.

2.4 Defining the KNN predict functions

```

1  # KNN predict
2
3  def knn_predict_regression(X_train, y_train, X_test, k=5):
4      preds = []
5
6      for x in X_test:
7          dists = euclidean_distances(X_train, x)
8          nn_idx = np.argsort(dists)[:k]
9          preds.append(np.mean(y_train[nn_idx]))
10
11     return np.array(preds)
12
13  def knn_predict_classification(X_train, y_train, X_test, k=5):
14      preds = []
15
16      for x in X_test:
17          dists = euclidean_distances(X_train, x)
18          nn_idx = np.argsort(dists)[:k]
19          votes = y_train[nn_idx]
20          most_common = Counter(votes).most_common()
21          preds.append(most_common[0][0])
22
23     return np.array(preds)

```

KNN predict regression

- Takes the following parameters:
 - `X_train`: training features
 - `y_train`: training targets
 - `X_test`: test features
 - `k`: number of nearest neighbours
- The `preds` list is an empty list which stores the predicted values.

- **For loop**
 - Euclidean distances are calculated from a particular test sample (x) to all training samples (X_{train}). These are stored in `dists`.
 - `nn_idx` is an array with the distances in descending order. `[:k]` takes the first k indices or nearest neighbours.
 - The y values of the k nearest neighbours are taken and their means are calculated for regression. This is then appended to `preds`.
- The `preds` list is returned as a NumPy array.

KNN predict classification

- Takes the following parameters:
 - `X_train`: training features
 - `y_train`: training targets
 - `X_test`: test features
 - `k`: number of nearest neighbours
- The `preds` list is an empty list which stores the predicted values.
- **For loop**
 - Euclidean distances are calculated from a particular test sample (x) to all training samples (X_{train}). These are stored in `dists`.
 - `nn_idx` is an array with the distances in descending order. `[:k]` takes the first k indices or nearest neighbours.
 - The `votes` list stores the exact labels of the nearest neighbours.
 - The `most_common` variable counts how many times each label appears and sorts them by frequency.
 - The label with the highest vote count is the predicted class. This is appended to `preds`.
- The `preds` list is returned as a NumPy array.

2.5 KNN regression

2.5.1 Defining the target

```
1 target = 'life_expec'
2 X = data.drop(columns=target).values
3 y_reg = data[target].values.astype(float)
```

- The `life_expec` variable is defined as the target for regression. This is stored in `target`.
- The feature matrix `X` is created by dropping the `life_expec` variable (stored in `target`). This leaves only the input features.
- `y_reg` creates the regression target vector.

2.5.2 Train test split

```
1 # split
2
3 X_train, X_test, y_train_reg, y_test_reg = train_test_split(X, y_reg, test_size=0.25, random_state=67)
```

Train test split is performed (75/25).

- `x`: full feature matrix is passed
- `y_reg`: target vector (`life_expec`)
- `test_size=0.25`: 25% of data goes to test set
- `random_state=67`: fixed random seed

2.5.3 Scaling

```
1 # scale
2
3 scaler = StandardScaler()
4 X_train_scaled = scaler.fit_transform(X_train)
5 X_test_scaled = scaler.transform(X_test)
```

- The `scaler` object creates an instance of the `StandardScaler` class.
- `X_train_scaled` is calculated by doing both fit and transform operations on `X_train`. It has an approximate mean of 0 and approximate standard deviation of 1.
- `X_test_scaled` is calculated by performing the transform operation on `X_test`.

2.5.4 Calculating errors

```
1 # errors
2
3 def rmse(a, b):
4     |     return np.sqrt(np.mean((a - b) ** 2))
5
6 def mae(a, b):
7     |     return np.mean(np.abs(a - b))
```

RMSE (root mean square error)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (a_i - b_i)^2}$$

MAE (mean absolute error)

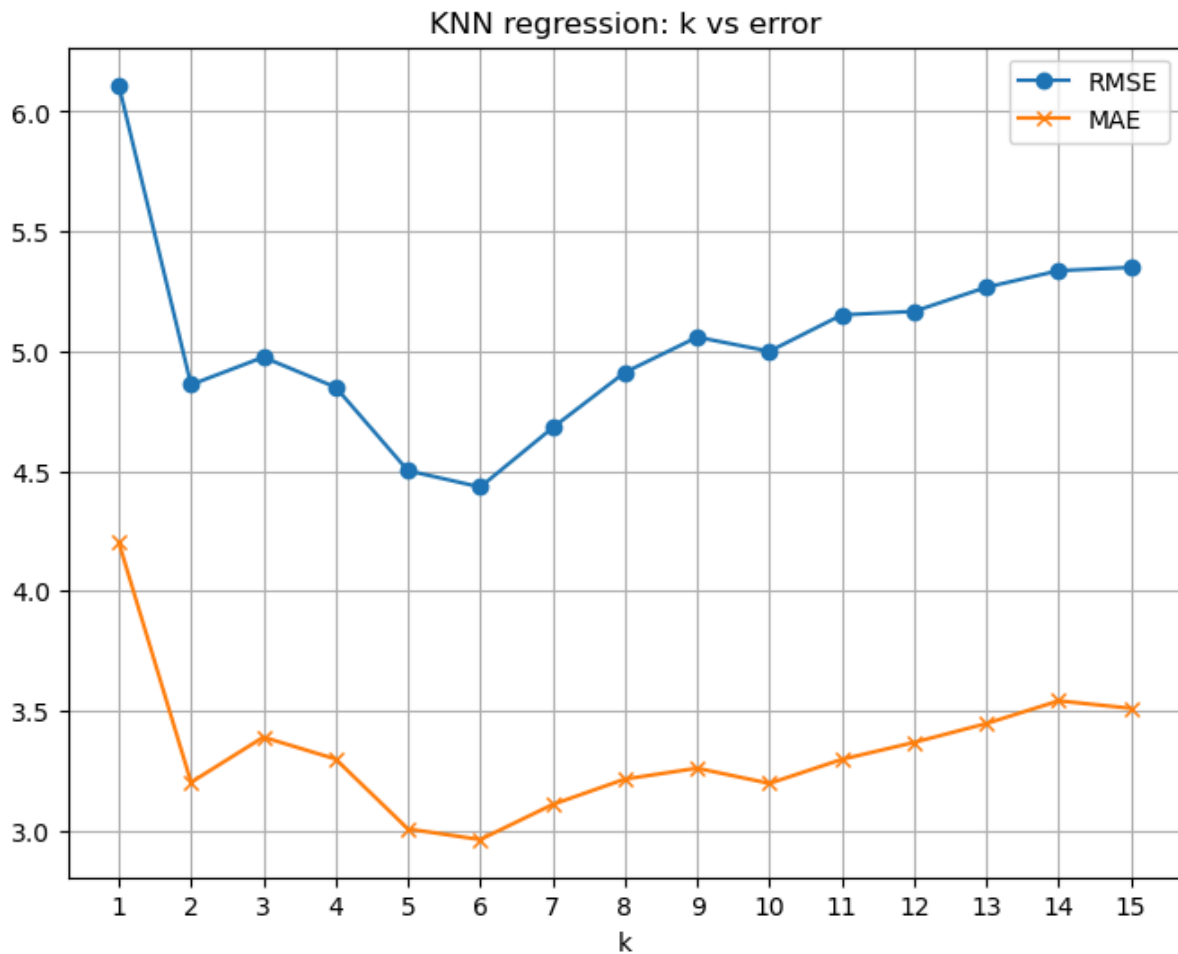
$$MAE = \frac{1}{n} \sum_{i=1}^n |a_i - b_i|$$

2.5.5 Finding best k by RMSE

```
1 # find best k by RMSE (1..15)
2
3 ks = list(range(1, 16))
4 rmse_vals = []
5 mae_vals = []
6
7 for k in ks:
8     pred = knn_predict_regression(X_train_scaled, y_train_reg, X_test_scaled, k=k)
9     rmse_vals.append(rmse(pred, y_test_reg))
10    mae_vals.append(mae(pred, y_test_reg))
11
12 best_k = ks[int(np.argmin(rmse_vals))]
13 best_rmse = min(rmse_vals)
14 print(f"Best k by RMSE: {best_k} (RMSE={best_rmse:.3f})")
✓ 0.0s
Best k by RMSE: 6 (RMSE=4.433)
```

- The `ks` list creates a list of candidate values for `k` (1, 2, 3,... 15).
- This is because
 - A small `k` value has low bias and high variance.
 - A large `k` value has high bias and low variance.
 - A reasonable range needs to be tested.
- The empty lists `rmse_vals` and `mae_vals` store the RMSE and MAE for each `k`.
- **For loop**
 - Generate predictions using the `knn_predict_regression` function.
 - Computes RMSE and MAE between predicted values and true values.
- The `best_k` value is the value of `k` which minimizes RMSE.
- The `best_rmse` value is the minimum RMSE value.
- We can see that the best `k` is 6 for an RMSE of 4.433.

2.5.6 k vs error graph



- **Legend**
 - **X-axis:** k (number of nearest neighbours)
 - **Y-axis:** error
 - **Blue curve:** RMSE
 - **Orange curve:** MAE
- Very small k performs poorly.
 - At $k = 1$, both RMSE (~ 6.1) and MAE (~ 4.2) are high.
 - At a single neighbour, the model is extremely sensitive to noise and outliers (high variance).
- Error decreases as k increases ($k = 2$ to 5 or 6)
 - RMSE and MAE drop sharply.
 - RMSE and MAE are minimum at $k \sim 6$ and $k \sim 5-6$ respectively.
 - The noise gets smoothed out because of more neighbours.
 - Bias-variance tradeoff improves.
- Optimal k is around 5-6.
- Large k leads to underfitting.
- The RMSE curve is steeper than MAE. This indicates the presence of some large errors/outliers.

2.5.7 Evaluate metrics

```
1 # evaluate metrics
2
3 pred_best = knn_predict_regression(X_train_scaled, y_train_reg, X_test_scaled, k=best_k)
4 print("Regression metrics on test split:")
5
6 # RMSE and MAE
7 print(f"RMSE: {rmse(pred_best, y_test_reg):.3f}")
8 print(f"MAE: {mae(pred_best, y_test_reg):.3f}")
9
10 # R2 score
11 ss_res = np.sum((y_test_reg - pred_best)**2)
12 ss_tot = np.sum((y_test_reg - np.mean(y_test_reg))**2)
13 r2 = 1 - ss_res/ss_tot
14 print(f"R2 score: {r2:.3f}")
```

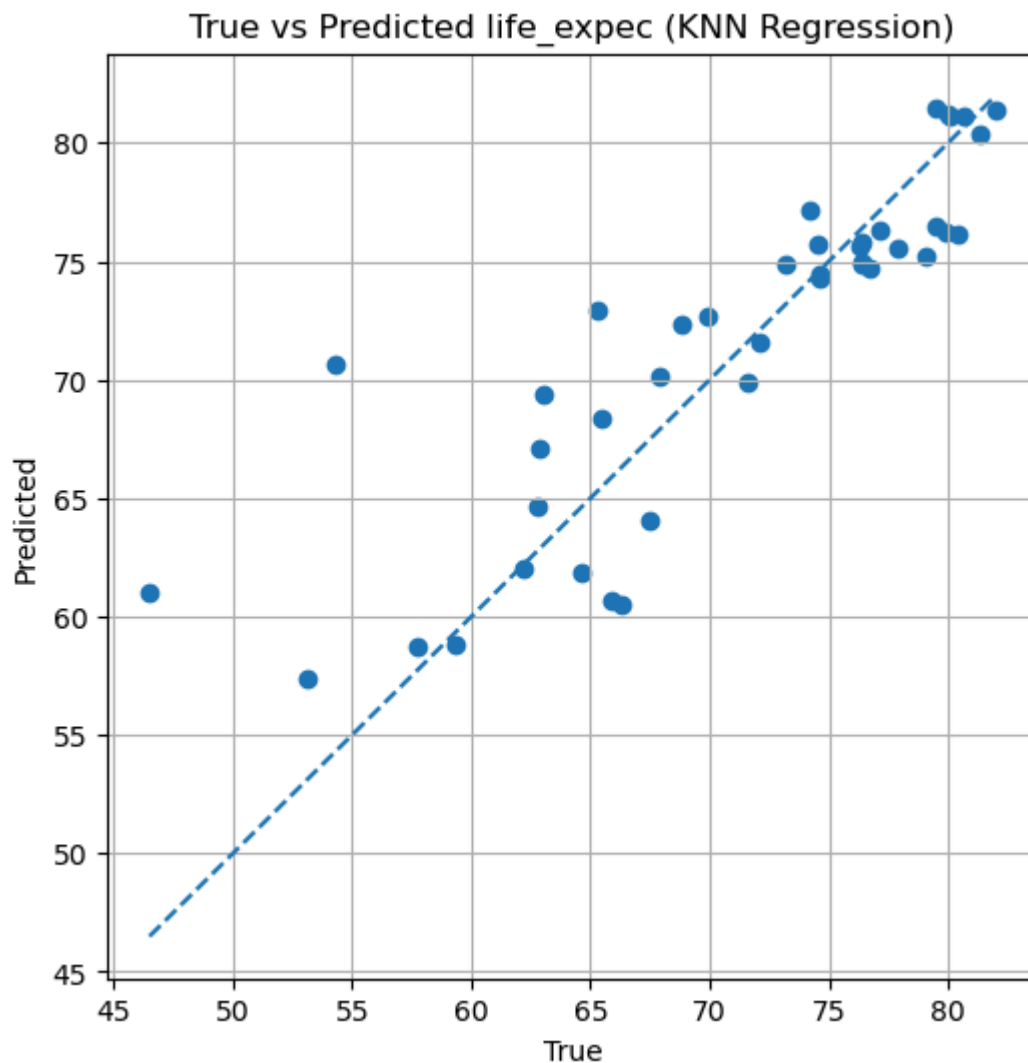
✓ 0.0s

- `pred_best` indicates the predictions for best `k`.
- RMSE and MAE are calculated using their respective functions.
- **R² score**
 - `ss_res` indicates the residual sum of squares.
 - `ss_tot` indicates the total sum of squares.
 - R² score is calculated by the formula

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

```
Regression metrics on test split:
RMSE: 4.433
MAE: 2.962
R2 score: 0.735
```

2.5.8 Regression plot



- **Legend**
 - **X-axis:** true life expectancy
 - **Y-axis:** predicted life expectancy using KNN regression
 - **Dashed line (y=x):** perfect predictions
- Most points lie close to the $y=x$ line. This indicates a good overall prediction accuracy.
- The error distribution is fairly symmetric. Points are scattered both above and below the diagonal. Well-calibrated model with no consistent over-predicting or under-predicting.
- Few visible outliers.

2.6 KNN classification

2.6.1 Creating binary labels using median

```
1 # create binary label using median
2
3 median_le = np.median(y_reg)
4 y_class = (y_reg >= median_le).astype(int) # 0: low life expectancy, 1: high
✓ 0.0s
```

- The `median_le` stores median life expectancy across all samples.
- The `y_class` is a boolean array as an integer, which means that 0 indicates a low life expectancy and 1 indicates a high life expectancy.

2.6.2 Train test split

```
1 # train test split
2
3 X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X, y_class, test_size=0.25, random_state=67)
✓ 0.0s
```

Train test split is performed (75/25).

- `X`: full feature matrix is passed
- `y_class`: binary labels
- `test_size=0.25`: 25% of data goes to test set
- `random_state=67`: fixed random seed

2.6.3 Scaling

```
1 # scale
2
3 scaler_c = StandardScaler()
4 X_train_c_scaled = scaler_c.fit_transform(X_train_c)
5 X_test_c_scaled = scaler_c.transform(X_test_c)
✓ 0.0s
```

- The `scaler_c` object creates an instance of the `StandardScaler` class.
- `X_train_c_scaled` is calculated by doing both fit and transform operations on `X_train_c`. It has an approximate mean of 0 and approximate standard deviation of 1.
- `X_test_c_scaled` is calculated by performing the transform operation on `X_test_c`.

2.6.4 Finding best k by accuracy

```
1 # find best k by accuracy (1..15)
2
3 ks = list(range(1, 16))
4 accs = []
5
6 for k in ks:
7     pred_c = knn_predict_classification(X_train_c_scaled, y_train_c, X_test_c_scaled, k=k)
8     acc = (pred_c == y_test_c).mean()
9     accs.append(acc)
10
11 best_k_c = ks[int(np.argmax(accs))]
12 best_acc = max(accs)
13 print(f"Classification: best k by accuracy = {best_k_c} (accuracy={best_acc:.3f})")
✓ 0.0s
Classification: best k by accuracy = 3 (accuracy=0.976)
```

- The `ks` list creates a list of candidate values for `k` (1, 2, 3,... 15).
- This is because
 - A small `k` value has low bias and high variance.
 - A large `k` value has high bias and low variance.
 - A reasonable range needs to be tested.
- The empty list `accs` stores the accuracy for each `k`.
- **For loop**
 - Generate predictions using the `knn_predict_classification` function.
 - Computes accuracy for each `k`.
- The `best_k_c` value is the value of `k` which gives maximum accuracy.
- The `best_acc` value is the maximum accuracy value.
- We can see that the best `k` is 3 for an accuracy of 0.976.

2.6.5 Evaluate metrics

Accuracy

```
1 # accuracy
2
3 def accuracy(y_true, y_pred):
4     return (y_true == y_pred).mean()
✓ 0.0s
```

Confusion matrix

```
1 # confusion matrix
2
3 def confusion_matrix(y_true, y_pred):
4     tp = np.sum((y_true == 1) & (y_pred == 1))
5     fp = np.sum((y_true == 0) & (y_pred == 1))
6     tn = np.sum((y_true == 0) & (y_pred == 0))
7     fn = np.sum((y_true == 1) & (y_pred == 0))
8     return np.array([[tn, fp], [fn, tp]])
```

✓ 0.0s

Precision, recall, F1 score

```
1 # precision, recall, f1 score
2
3 def f1(y_true, y_pred, pos_class):
4     tp = np.sum((y_true == pos_class) & (y_pred == pos_class))
5     fp = np.sum((y_true != pos_class) & (y_pred == pos_class))
6     fn = np.sum((y_true == pos_class) & (y_pred != pos_class))
7
8     precision = tp / (tp + fp) if (tp + fp) != 0 else 0
9     recall = tp / (tp + fn) if (tp + fn) != 0 else 0
10    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) != 0 else 0
11
12    return precision, recall, f1
```

✓ 0.0s

Classification report

```
1 # overall classification report
2
3 def classification_report(y_true, y_pred):
4     cm = confusion_matrix(y_true, y_pred)
5     tn, fp = cm[0]
6     fn, tp = cm[1]
7
8     print("Confusion Matrix:")
9     print(cm)
10    print()
11
12    prec0, rec0, f10 = f1(y_true, y_pred, pos_class=0)
13    prec1, rec1, f11 = f1(y_true, y_pred, pos_class=1)
14
15    print("Class 0:")
16    print(f"Precision: {prec0:.3f}")
17    print(f"Recall: {rec0:.3f}")
18    print(f"F1 score: {f10:.3f}\n")
19
20    print("Class 1:")
21    print(f"Precision: {prec1:.3f}")
22    print(f"Recall: {rec1:.3f}")
23    print(f"F1 score: {f11:.3f}\n")
24
25    acc = accuracy(y_true, y_pred)
26    print(f"Overall accuracy: {acc:.3f}")
```

✓ 0.0s

- The `cm` variable indicates the confusion matrix.
- `prec0`, `rec0`, `f10` are the precision, recall, F1 score for the positive class 0.
- `prec1`, `rec1`, `f11` are the precision, recall, F1 score for the positive class 1.
- These metrics are then printed.
- The overall accuracy is stored in `acc`.

```
1 pred_c_best = knn_predict_classification(X_train_c_scaled, y_train_c, X_test_c_scaled, k=best_k_c)
2 classification_report(y_test_c, pred_c_best)
✓ 0.0s
```

- `pred_c_best` runs KNN classification on the best `k` (`best_k_c`).
- We then print the classification report.

Confusion Matrix:

```
[[19  1]
 [ 0 22]]
```

Class 0:

Precision: 1.000

Recall: 0.950

F1 score: 0.974

Class 1:

Precision: 0.957

Recall: 1.000

F1 score: 0.978

Overall accuracy: 0.976

3. K-Means Clustering

```
1 X_orig = data.values.astype(float)
✓ 0.0s
```

X_orig is a NumPy array from the DataFrame data.

3.1 Scaling

```
1 # scaler
2
3 class SimpleScaler:
4     def fit(self, X):
5         X = np.asarray(X)
6         self.mean_ = X.mean(axis=0)
7         self.std_ = X.std(axis=0, ddof=0)
8         self.std_[self.std_ == 0] = 1
9
10    def transform(self, X):
11        return (X - self.mean_) / self.std_
12
13    def fit_transform(self, X):
14        self.fit(X)
15        return self.transform(X)
16
17    def inverse_transform(self, Xs):
18        return Xs * self.std_ + self.mean_
✓ 0.0s
```

- Everything in SimpleScaler is the same as StandardScaler. The only new addition is the `inverse_transform()` function.
- The `inverse_transform()` function reverts the scaled data back to the original state.

3.2 Calculating distances

```
1  # distances
2
3  def pairwise_squared_distances(a, b):
4      a = np.asarray(a)
5      b = np.asarray(b)
6
7      aa = (a ** 2).sum(axis=1)[:, None]
8      bb = (b ** 2).sum(axis=1)[None, :]
9      ab = a.dot(b.T)
10
11     return aa + bb - 2*ab
```

✓ 0.0s

- The `pairwise_squared_distances()` function calculates squared Euclidean distances.
- It is ensured that both `a` and `b` are NumPy arrays.
- `aa` and `bb` store the squared norm of each row in `a` and `b`. They are reshaped to `(n_a, 1)` and `(1, n_b)` respectively.
- `ab` stores the dot product of `a` and `b` transpose. The resulting shape is `(n_a, n_b)`.
- Returns using the identity

$$\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2(a \cdot b)$$

3.3 Defining helper functions

3.3.1 Initializing centroids

```
1 # initialize centroids
2
3 def init_centroids(X, k, random_state=None, method='kmeans++'):
4     rng = np.random.RandomState(random_state)
5     n = X.shape[0]
6
7     if method == 'random':
8         idx = rng.choice(n, size=k, replace=False)
9         return X[idx].copy()
10
11     centers = []
12     idx0 = rng.randint(0, n)
13     centers.append(X[idx0].copy())
14
15     for _ in range(1, k):
16         d2 = np.min(pairwise_squared_distances(X, np.vstack(centers)), axis=1)
17         probs = d2 / d2.sum()
18         r = rng.rand()
19         cum = np.cumsum(probs)
20         next_idx = np.searchsorted(cum, r)
21         centers.append(X[next_idx].copy())
22
23     return np.vstack(centers)
```

✓ 0.0s

- Defines a function to initialize k centroids from the dataset X.
- **Parameters:**
 - X: data matrix
 - k: number of clusters
 - random_state: seed for reproducibility
 - method: 'random' or 'kmeans++'
- The rng object is a pseudorandom number generator.
- n stores the number of data points.
- **Random initialization block**
 - If the method is 'random', the function randomly selects k distinct indices.
 - It uses the corresponding rows of X as centroids.
- The centers list is an empty list used to store chosen centroids.
- idx0 randomly selects the first centroid and adds it to the centers list. This is because kmeans++ always starts with one random point.
- **For loop**
 - Loop to select the remaining k-1 centroids.
 - The squared distance is computed from each point in X to each existing centroid. This is stored in d2.
 - The distances are converted into a probability distribution probs.

- `r` draws a random number in $[0, 1)$.
 - The `cum` array creates a cumulative probability array.
 - `next_idx` finds the index where `r` fits into `cum`.
 - The newly selected centroid is added to `centers`.
- The returned array is a vertical stack of `centers`. It is stacked into the shape $(k, n_features)$.

3.3.2 Assign clusters

```

1  # assign clusters
2
3  def assign_clusters(X, centers):
4      d2 = pairwise_squared_distances(X, centers)
5      labels = np.argmin(d2, axis=1)
6      inertia = np.sum(np.min(d2, axis=1))
7      return labels, inertia
✓ 0.0s

```

- **Parameters:**
 - `X`: data points
 - `centers`: current cluster centroids
- Computes squared Euclidean distances between every point in `X` and every centroid in `centers`. This is stored in `d2`.
- The `labels` variable assigns each point to the nearest centroid.
- `inertia` stores the within-cluster sum of squares.
- Returns `labels` and `inertia`.

3.3.3 Update centers

```

1  # update centers
2
3  def update_centers(X, labels, k, random_state=None):
4      n_features = X.shape[1]
5      centers = np.zeros((k, n_features))
6      rng = np.random.RandomState(random_state)
7
8      for i in range(k):
9          members = X[labels == i]
10         if len(members) == 0: centers[i] = X[rng.randint(0, X.shape[0])]
11         else: centers[i] = members.mean(axis=0)
12
13     return centers
✓ 0.0s

```

- **Parameters:**
 - `X`: data matrix
 - `labels`: cluster index for each point
 - `k`: number of clusters
 - `random_state`: seed for reproducibility
- `n_features` stores the number of features per data point.
- `centers` initializes a placeholder for updated centroids.
- `rng` is a pseudorandom number generator.
- **For loop**
 - Loops over each cluster.
 - `members` selects all points assigned to cluster `i`.
 - If a cluster is empty, it reinitializes its center to a random data point.
 - Otherwise, it computes the mean of all points in cluster `i`.

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

- Returns the updated centroids.

3.4 Defining the K-Means function

```

1 # K-Means
2
3 def kmeans(X, k, max_iter=300, tol=1e-4, random_state=None, init_method='kmeans++'):
4     X = np.asarray(X)
5     centers = init_centroids(X, k, random_state=random_state, method=init_method)
6
7     for i in range(1, max_iter+1):
8         labels, inertia = assign_clusters(X, centers)
9         new_centers = update_centers(X, labels, k, random_state=random_state)
10        max_shift = np.sqrt(((centers - new_centers)**2).sum(axis=1)).max()
11        centers = new_centers
12        if max_shift <= tol: break
13
14    labels, inertia = assign_clusters(X, centers)
15    return {'centers': centers, 'labels': labels, 'inertia': float(inertia), 'n_iter': i}

```

✓ 0.0s

- **Parameters:**
 - `X`: data matrix
 - `k`: number of clusters
 - `max_iter`: maximum iterations
 - `tol`: convergence tolerance
 - `random_state`: seed for reproducibility
 - `init_method`: 'random' or 'kmeans++'
- It is ensured that `X` is a NumPy array for numerical operations.
- Initializes centroids using random sampling or k-means++.
- **Optimization for loop**
 - Runs K-Means iterations until convergence or max iterations.
 - Assigns each point to the nearest centroid and computes inertia.

- Recomputes centroids as the mean of assigned points.
- Computes the maximum centroid movement.
- The centroids are updated for the next iteration.
- Stops early if centroids move less than tolerance.
- Final assignment of labels and inertia.
- Returns
 - `centers`: final centroids
 - `labels`: cluster assignment for each point
 - `inertia`: final clustering error
 - `n_iter`: number of iterations taken

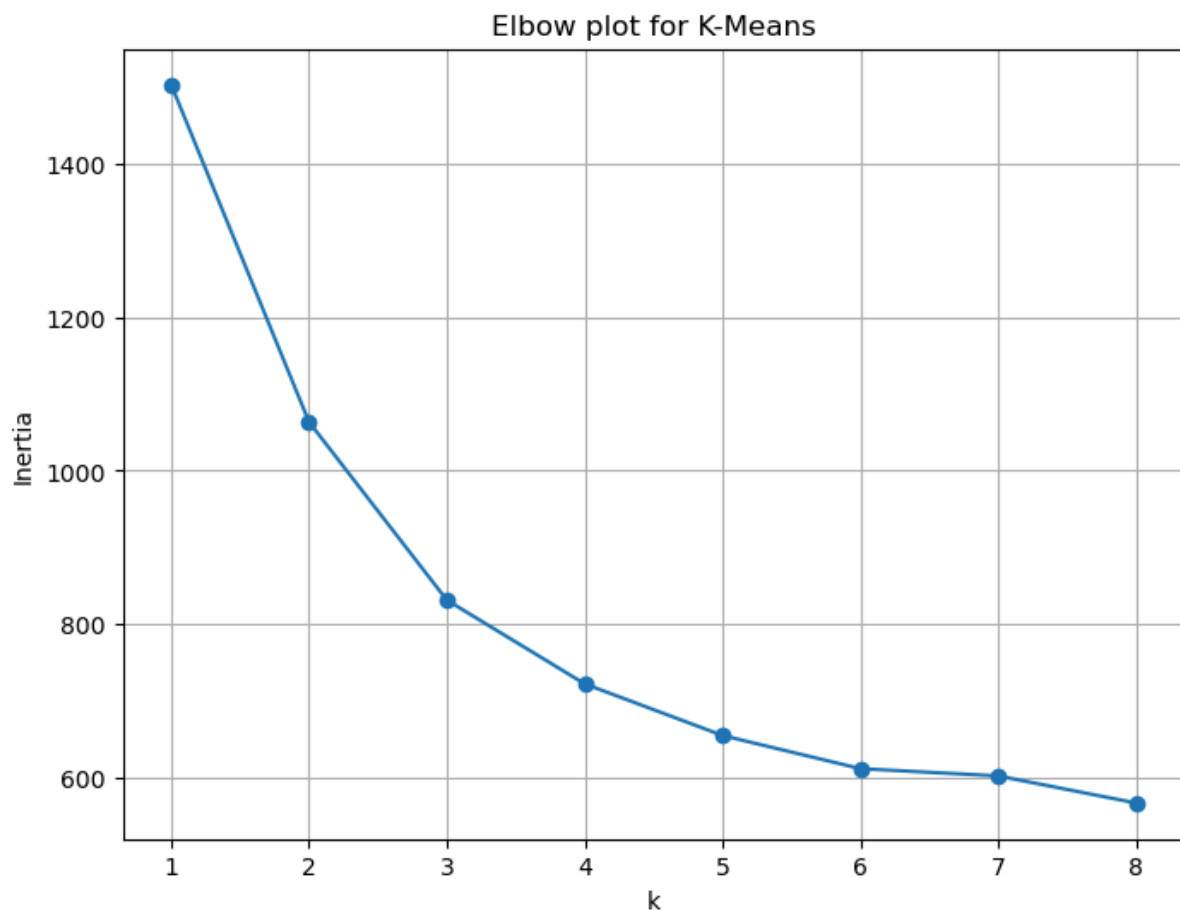
3.5 Initialize scaler

```
1  # initialize scaler
2
3  scaler = SimpleScaler()
4  X = scaler.fit_transform(X_orig)
```

✓ 0.0s

- The `scaler` object is an instance of the `SimpleScaler` class.
- `X` is the scaled version of `X_orig`.

3.6 Elbow plot



- **Legend:**
 - **X-axis:** k (number of clusters)
 - **Y-axis:** inertia (within-cluster sum of squared distances)
- Inertia always decreases as k increases.
- Sharp decrease for small k.
 - Big drop from k = 1, 2, 3
 - Indicates that adding clusters up to this point greatly improves compactness.
- Elbow around k = 3 or 4.
 - After k = 3~4, the curve starts to flatten.
 - Marginal improvement beyond this is small.
 - This bend is the elbow point.
- Diminishing returns for larger k.
 - From k = 5 onward, inertia decreases slowly.
 - Extra clusters mostly over-segment the existing structure.
- The best choice of k is around 3 or 4.

3.7 Printing inertia for each k

```
k → inertia
1 → 1503.000
2 → 1063.483
3 → 831.424
4 → 722.300
5 → 655.030
6 → 611.805
7 → 602.297
8 → 567.071
```

3.8 Choose best k

```
1 # choose k
2
3 chosen_k = 3
4 res = results[chosen_k]
5 labels = res['labels']
6 centers_scaled = res['centers']
7 centers_orig = scaler.inverse_transform(centers_scaled)
```

- From the elbow plot, our `chosen_k` is 3.
- `res` stores the K-Means result corresponding to `k = 3`.
- `labels` extracts cluster assignments.
- `centers_scaled` extracts centroids in the scaled feature space.
- `centers_orig` transforms centroids back to the original feature scale.

3.9 Defining cluster sizes

```
1 # cluster sizes
2
3 unique, counts = np.unique(labels, return_counts=True)
4 cluster_sizes = dict(zip(map(int, unique), map(int, counts)))
5
6 print(f"Chosen k: {chosen_k}")
7 print(f"Cluster sizes: {cluster_sizes}")
✓ 0.0s
```

- `labels` is an array of cluster assignments.
- `np.unique` sorts unique cluster IDs.
- `counts` is the number of samples in each cluster.
- `zip()` pairs cluster ID with its count.
- `dict()` converts pairs into a dictionary.
- `map(int,)` ensures plain Python integers and not NumPy types.
- Chosen `k` and cluster sizes are printed.

```
Chosen k: 3
Cluster sizes: {0: 47, 1: 36, 2: 84}
```

3.10 Displaying centers

```
1 # display centers as df
2
3 centers = pd.DataFrame(centers_orig, columns=data.columns)
4 centers.index.name = 'cluster'
✓ 0.0s
```

- Converts the `centers` array to a DataFrame.
- The row index is `'cluster'`.

```
1 centers.round(4)
✓ 0.0s
```

	child_mort	exports	health	imports	income	inflation	life_expec	total_fer	gdpp
cluster									
0	92.9617	29.1513	6.3885	42.3234	3942.4043	12.0197	59.1872	5.0081	1922.3830
1	5.0000	58.7389	8.8078	51.4917	45672.2222	2.6712	80.1278	1.7528	42494.4444
2	21.9274	40.2439	6.2010	47.4734	12305.5952	7.6009	72.8143	2.3075	6486.4524